CEN3024C Module 6 Project

For this project we will be implementing a CORS policy at the global level and a "one-off" method-level policy to the Task Manager project. We will also be adding support for password hashing for the users. To support this the data.sql script must be modified, use the new version provided at the top level of the project in the starter repo after implementing the new hash and salt fields.

1. Add a global policy using a configuration class (add this in a new "config" package):

```
package edu.fscj.cen3024c.taskmanager.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
                .allowedOrigins("http://example.com")
                .allowedMethods("GET", "POST", "PUT", "DELETE")
                .allowedHeaders("*")
                .allowCredentials(true);
    }
}
```

2. Choose an endpoint in your controllers and add a method-based policy:

```
@CrossOrigin(origins = {"http://example.com", "http://localhost"})
@GetMapping
public List<UserDTO> getAllUsers() {
    return userService.findAll();
}
```

I added mine to the “get all” endpoint in the User class. Technically it doesn’t matter if the @CrossOrigin annotation preceeds or follows the @GetMapping annotation, but convention dictates that it should go before @GetMapping.

3. Test and verify the correct operation of your policies with curl by specifying the Origin field. Any request using the example.com origin should return a 200 and display the corresponding data, e.g.,

curl -i -H "Origin: http://example.com" -X GET http://localhost:8080/users
...
HTTP/1.1 200

But a request from example2 should not:

curl -i -H "Origin: http://example2.com" -X GET http://localhost:8080/users
...

HTTP/1.1 403
Invalid CORS request

4. Add a salt and hash field to the user entity and remove the existing password field. Note that the password field in the DTOs remains, we are now salting and hashing it for persistent storage and no longer storing a cleartext password in the database.

   Modify the UserService class to implement the salt and hash operations using PBKDF2WithHmacSHA1 (a popular algorithm recommended by NIST), or you can try a different algorithm if you like.

```
@Transactional
public UserDTO createUser(CreateUserRequestDTO request) {

    String salt = generateSalt();

    String hash;
    try {
        hash = hashPassword(request.getPassword(), salt);
    } catch (Exception e) {
        throw new RuntimeException("Error hashing password", e);
    }

    User user = new User();
    user.setUsername(request.getUsername());

    // NEW FIELDS ON ENTITY (you must add these in User.java)
    user.setSalt(salt);
    user.setPasswordHash(hash);

    User savedUser = userRepository.save(user);
    return convertToDTO(savedUser);
}

@Transactional
public UserDTO updateUser(Integer id, UpdateUserRequestDTO request) {

    User existingUser = findByIdEntity(id);

    existingUser.setUsername(request.getUsername());

    // Always re-hash since password is required
    String salt = generateSalt();

    String hash;
    try {
        hash = hashPassword(request.getPassword(), salt);
    } catch (Exception e) {
        throw new RuntimeException("Error hashing password", e);
    }
```

```
        existingUser.setSalt(salt);
        existingUser.setPasswordHash(hash);

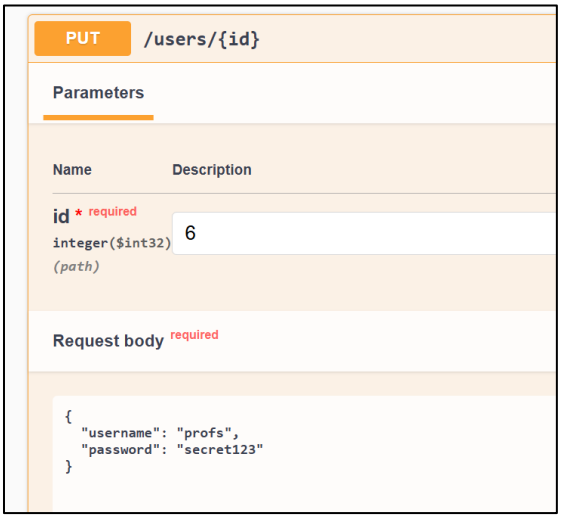
        User updatedUser = userRepository.save(existingUser);
        return convertToDTO(updatedUser);
    }
```

Use Swagger to add a user and then update them; verify that your PUT and POST perform the salt and hash operations and save the values to the database (I did this by using the IntelliJ DB tool, simple clicking on the user table displays the contents:

id	password…	salt	usern…
1	BBBBBBBBBBBBBBBB…	AAAAAAAAAAAAAAAA…	alice
2	BBBBBBBBBBBBBBBB…	AAAAAAAAAAAAAAAA…	bob
3	BBBBBBBBBBBBBBBB…	AAAAAAAAAAAAAAAA…	carol
4	BBBBBBBBBBBBBBBB…	AAAAAAAAAAAAAAAA…	dave
5	BBBBBBBBBBBBBBBB…	AAAAAAAAAAAAAAAA…	eve
6	gdIBWwndePwWDk8J5…	8r8tZO0VjL0+ivzTu…	ProfS

Note that the revised data.sql file inserts dummy values for the prepopulated users, we would want to ensure these users were removed before deploying our application to production.

To test the Put, you will need to specify the unique ID for the table (e.g., 6 from the above data). Change the password also:



After changing the user's information, the salt and hash should change even if the password does not change. In a production system, we would typically make the password optional in the update DTO and rehash only if it was changed; for this design we are keeping it simple and just rehashing every time we update.

id	password…	salt	user
1	BBBBBBBBBBBBBBBB…	AAAAAAAAAAAAAAAA…	alice
2	BBBBBBBBBBBBBBBB…	AAAAAAAAAAAAAAAA…	bob
3	BBBBBBBBBBBBBBBB…	AAAAAAAAAAAAAAAA…	carol
4	BBBBBBBBBBBBBBBB…	AAAAAAAAAAAAAAAA…	dave
5	BBBBBBBBBBBBBBBB…	AAAAAAAAAAAAAAAA…	eve
6	47JHSxVK+HtLF1/8J…	63dhx1UzrWZ3BgurC…	profs

5. Submit your solution to the GitHub classroom assignment repo. Your screenshots should include the results of the CORS test and demonstration of the password salt/hashing operations for User POST and PUT.